

CCOP API 통합 레퍼런스

전체 40개 엔드포인트(`/api/v1/*` 37 + 외부 read-only `/api/v1/graph/{read,dump,schema}` 3)의 코드 실측 레퍼런스. 기능 그룹별 정리.

작성: 2026-07-15 (dev `58bf8b1` 기준, 코드에서 실측) · Base URL: `https://<HOST>/api/v1` 관련 문서: [파트너 온보딩 API_GUIDE.md](#) · 외부 조회 [EXTERNAL_GRAPH_QUERY_GUIDE.md](#) · [EXTERNAL_CYPHER_QUERY_HOWTO.md](#)

Blueprint 구성

Blueprint	prefix	파일	엔드포인트
<code>api_v1</code>	<code>/api/v1</code>	<code>app/routes_api.py</code>	37
<code>graph_read_bp</code>	<code>/api/v1/graph</code>	<code>app/routes_graph_read.py</code>	3 (외부 read-only)

1. 인증

두 가지 인증 방식이 공존한다. 엔드포인트마다 어느 쪽인지(또는 무인증인지) 반드시 확인할 것.

1.1 Bearer 토큰 — 파트너 API (`require_api_key` , `app/middleware/api_auth.py:159`)

- 헤더: `Authorization: Bearer <KEY>` (키는 `ccop_` 접두사, 기관별 발급)
- 검증: **SHA-256** 해시로 저장소(`data/api_keys.json`) 조회, `is_active` 확인
- 실패: **401**(헤더 없음/형식오류) · **403**(무효 키)
- 성공 시 `request.partner` , `request.partner_data` 주입
- 추가 데코레이터 `require_endpoint_permission('<ep>')` : 티어의 `allowed_endpoints` 에 없으면 **403** (현재 `graph-query` 에만 적용)

1.2 X-API-Key — 외부 read-only 그래프 API (`_check_api_key` , `app/routes_graph_read.py:37`)

- 헤더: `X-API-Key: <TOKEN>` (운영팀 제공 **단일 조회 토큰**, 기관별 키 아님)
- 검증: 환경변수 `LLM_API_KEY` 와 `hmac.compare_digest` (timing-safe) 비교
- **fail-closed**: `LLM_API_KEY` 미설정 시 외부 조회 전면 거부
- 실패: **401** `{"error": "Invalid API key"}`

1.3 티어 / Rate limit (`app/models/api_key.py:127`)

티어	rate_limit	max_results	allowed_endpoints
free	1000	50	text-to-cypher, usage
startup	10000	100	text-to-cypher, graph-query, usage, validate-cypher
enterprise	무제한(None)	500	* (전체)

- ⚠ **Rate limit 단위 주의**: 저장값(1000/10000)은 코드·문서엔 "시간당/월"로 적혀 있으나 **실제 강제는 분당 버킷** (`api_auth.py:126`)이다. 초과 시 **429**. 통합 관점에서는 "분당 한도"로 이해할 것. (문서 정합 정리 필요 — §7 개선 항목)

1.4 공통 에러 형식

표준 에러 헬퍼는 없고 두 형태가 혼재: 최소 `{"error": "..."}` , 인증/미들웨어 계열은 `{"error": "...", "message": "..."}` .

코드	의미
400	필수 파라미터 누락·잘못된 값
401	인증 헤더 없음/키 무효
403	read-only 위반·권한(티어) 부족
429	rate limit 초과
404	리소스 없음(사건/워크플로)
500	내부 오류

1.5 ⚠️ 보안 주의 (코드 실측 — 인수인계 필수 확인)

- ✅ [해결됨] `graph/*` mutating 그룹(create·delete·node/create·edge/create·element/delete)의 무인증 노출은 인증 복구 완료 (create/delete 등 admin 권한 필요, graph/list Bearer). `_check_rate_limit` 의 `rate_limit=None` TypeError 도 수정. → `fix/graph-admin-auth`, `test_security.py::TestGraphAdminAuth`
- ✅ [해결됨] UI-소비 v1 엔드포인트(`network/*`, `styles`, `etl/analyze`, `gdb`, `pipeline`, `workflows`)의 무인증/헤더 불일치 (`demo_key_123`)는 `require_api_or_ui` 이중 인증으로 정비 — same-origin UI 세션 쿠키 OR 파트너 Bearer 키. 하드코딩 데모키 제거, 깨져 있던 network 기능 복구. → `fix/ui-auth-session`
- 운영 배포 시 nginx 레벨 접근통제 또는 앱 Basic Auth(`BASIC_AUTH_USER/PASS` 설정 시 `/api/v1/health` 제외 전 경로 앞단 보호)로 감싸는 것을 전제로 설계됨.

2. Text2Cypher / 쿼리

엔드포인트	인증	요청	응답(200)	설명
POST /text-to-cypher	Bearer	question * (str), schema.graph_path (선택, 기본 tccop_graph_v6)	{status, cypher, intent, elements, results_count, partner, response_time_ms}	자연어→Cypher 변환 (LangGraph: reflection+schema fetch)
POST /agentic-query	Bearer	question *, graph_path (선택)	{status, agent_response(전체 result), partner, response_time_ms}	분석 에이전트 원시 결과 반환
POST /graph-query	Bearer + graph-query 권한	cypher *, graph_path (선택)	{status, results, count, limited, graph_path, response_time_ms}	Cypher 실행(읽기 전용; 쓰기 키워드 403, 티어 max_results 절삭)
POST /validate-cypher	Bearer	cypher *	{status, is_safe, warnings[], cypher}	실행 없이 문법·안전성 정적 검증

* = 필수(누락 시 400). 기본 graph_path = tccop_graph_v6 .

3. 그래프 CRUD / 조회

✅ 2026-07 하드닝 반영: 이전엔 무인증 노출이었으나 인증 복구됨. mutating 작업은 admin 권한(enterprise *****) 필요. UI는 이 v1 경로가 아닌 `routes.py` 의 `/api/graph/*` 를 사용하므로 영향 없음.

엔드포인트	인증	요청	응답(200)	설명
GET <code>/graph/list</code>	Bearer	—	<code>{status, graphs}</code>	그래프 목록
POST <code>/graph/create</code>	Bearer + admin	<code>graph_name</code> *	<code>{status, message}</code>	그래프 생성
POST <code>/graph/delete</code>	Bearer + admin	<code>graph_name</code> *	<code>{status, message}</code>	그래프 삭제 (파괴적)
POST <code>/graph/node/create</code>	Bearer + admin	<code>graph_name</code> , <code>label</code> , <code>properties</code> (기본 {})	<code>{status, node_id}</code>	수동 노드 추가
POST <code>/graph/edge/create</code>	Bearer + admin	<code>graph_name</code> , <code>src_id</code> , <code>tgt_id</code> , <code>label</code> , <code>properties</code> (기본 {})	<code>{status, edge_id}</code>	수동 엣지 추가
POST <code>/graph/element/delete</code>	Bearer + admin	<code>graph_name</code> , <code>element_id</code> , <code>is_edge</code> (기본 false)	<code>{status, message}</code>	노드/엣지 삭제

`admin = require_endpoint_permission('admin');` `allowed_endpoints` 에 ***** 인 enterprise 티어만 통과(free/startup 403).

4. 분석 (패턴 · 증거 · 네트워크)

엔드포인트	인증	요청	응답(200)	설명
POST /analyze-pattern	Bearer	case_id *, graph_path (선택)	{success, case_id, matched_patterns[], primary_pattern, confidence, analysis_summary}	사건의 범죄 패턴 자동 인식(패턴無 시 success:false)
GET /evidence-completeness/<case_id>	Bearer	path case_id, query graph_path	{success, **completeness(score, missing_evidence...)}	증거 완성도 평가 (사건無 404)
GET /patterns	Bearer	—	{patterns:[{pattern_id, name, description, required_nodes, required_edges, min_threshold}], total}	지원 범죄 패턴 목록
POST /network/project	세션/Bearer	graph_path, actor_label (기본 vt_psn), pivot_label (기본 vt_bacnt), min_shared (기본 1), projection_edge	{status, mode:"1mode", nodes[], edges[], stats}	2-mode→1- mode 투영(공유 pivot 기반 공범망, LIMIT 200)
POST /network/bipartite	세션/Bearer	graph_path, actor_label, pivot_label	{status, mode:"2mode", actor_count, pivot_count, edge_count, top_actors[], top_pivots[]}	이분 그래프 degree 분포 통계

network/* 는 actor/pivot 라벨 화이트리스트 검증(밖이면 400), 두 라벨 동일 시 400.

5. ETL / 파이프라인

etl/analyze-pipeline 은 UI 소비 → 세션/Bearer(require_api_or_ui). etl 확장/추론 import 변형은 여전히 무인증(\$13 후속). rdb/to-graph 는 Bearer.

엔드포인트	인증	요청	응답(200)	설명
POST /etl/analyze	세션/Bearer	multipart file *(.csv)	analyze_csv 결과 (columns/relationships/suggested_mappings)	CSV 자동 관계 추론
POST /etl/infer-import	없음	multipart file *, graph (기본 tccop_graph_v6), mapping (JSON, 선택)	{status, nodes_created, edges_created, graph, mapping_used}	추론/지정 매핑으로 적재
POST /etl/analyze-extended	없음	multipart file *(.csv)	{status, columns, row_count, action_detection, **result}	KICS 4- Layer 확장 스키마 분석
POST /etl/import-extended	없음	multipart file *, graph (기본 tccop_graph_v6)	{status, graph, action_nodes, entity_nodes, relationships, mapping}	확장 스키마 그래프 적재 (Action 노드 생성)
POST /rdb/to-graph	Bearer	graph_name (기본 test_ai01)	{status, message, stats}	RDB→ 그래프 변환
POST /pipeline/csv_to_v40_graph	세션/Bearer	multipart file / files *, graph_name (기본 v40_pipeline_demo), source_domain (기본 KICS), source_id (선택)	{status, pipeline, graph_name, source_domain, target_schema, elapsed_sec, layers{L1..L5}}	CSV→RDB→ 그래프→ 시각화 L1~L5 통합

source_domain 허용: KICS/OSINT/DIGITAL/EXT/INVESTIGATION/PARTNER/INFERENCE (밖이면 400).

6. RDB 조회

⚠️ 전부 무인증.

엔드포인트	요청	응답(200)	설명
GET /rdb/tables	—	<code>{status, tables:[{name, label, icon}]}</code>	RDB 테이블 목록(12종)
GET /rdb/stats	query <code>graph_name</code> (기본 test_ai01)	<code>{status, stats:{rdb{...}, gdb{graphs, graph_count, nodes, edges}}}</code>	RDB+GDB 통합 통계 (대시보드)
GET /rdb/query/<table_name>	path <code>table_name</code> (화이트리스트 18종), query <code>limit</code> (기본 50, 최대 500), <code>offset</code> (기본 0), <code>search</code>	<code>{status, table, columns, data[], total, limit, offset}</code>	테이블 페이징 ·검색 조회
GET /gdb/detail-stats (세션/Bearer)	query <code>graph_name</code> (기본 test_ai01)	<code>{status, data:{nodes:[{label,count}], edges:[{type,count}], total_nodes, total_edges}}</code>	GDB 라벨/ 타입별 상세 통계

7. 법률 RAG (`legal/*`) — 전부 Bearer

엔드포인트	요청	응답(200)	설명
<code>POST</code> <code>/legal/search</code>	<code>question</code> *(≤2000자), <code>top_k</code> (기본 5, 1~20), <code>mode</code> (hybrid bm25 vector, 기본 hybrid), <code>rerank</code> (bool 선택)	<code>{status, mode_used,</code> <code>rerank_used, results[]}</code>	법률 근거 하이브리드 검색 (점수 분해, 답변 없음)
<code>POST</code> <code>/legal/answer</code>	<code>question</code> *(≤2000자), <code>top_k</code> (기본 4, 1~10)	<code>{status, success, answer,</code> <code>citations[]}</code>	근거 인용[n] 자문 답변 + 비자문 고지
<code>GET</code> <code>/legal/status</code>	—	<code>{status, index_loaded,</code> <code>chunks, embedding_backend,</code> <code>db{...}}</code>	RAG 인덱스/ 임베딩/DB 적재 현황

잘못된 mode/top_k/rerank 타입은 400. 설계: `LEGAL_RAG_V2_DESIGN.md` .

8. 메타 / 스타일 / 워크플로 (GET)

visual-style·edge-style·layout-presets·workflows·workflows/execute 는 UI 소비 → 세션/Bearer(require_api_or_ui).
ontology/meta-schema/layers 는 아직 무인증 (§13 후속).

엔드포인트	요청	응답(200)	설명
GET /ontology/meta	—	{status, version:"v4.0", node_id_standard, domain_usage, inference_rules}	V4.0 온톨로지 메타 (ID표준/도메인/ 추론규칙)
GET /schema/layers	—	{layers, entities{ }, relationships{ }, entity_count, relationship_count}	KICS 4-Layer 스키마/ 엔티티/관계
GET /visual-style	query label (선택)	label有 {status, label, style} / 無 {status, count, styles}	V4.0 노드 시각화 표준
GET /edge-style	query edge (선택)	동일 구조(styles=EDGE_STYLE_V40)	V4.0 엣지 시각화 표준
GET /layout-presets	query name (선택)	동일 구조(presets 5종)	그래프 레이아웃 프리셋
GET /workflows	query name (선택)	동일 구조(workflows 6종)	수사 워크플로 정의
GET /workflows/<name>/execute	path name (6종), query graph_path (기본 tccop_v40_demo), limit (기본 200)	{status, workflow, graph_path, node_count, edge_count, elements[]}	사전정의 워크플로 실행 →elements(미정의 404)

워크플로 6종: case_to_suspects , suspect_to_assets , phishing_campaign_view , fund_flow , relay_station_network ,
cross_graph_sameAs .

9. 통계 / 관리

엔드포인트	인증	요청	응답(200)	설명
GET /usage	Bearer	—	<code>{partner, tier, current_month:{requests, limit, remaining}, breakdown{}, allowed_endpoints[]}</code>	파트너 월간 사용량/한도
GET /health	없음	—	<code>{status:"healthy", version:"1.0.0", service}</code>	헬스 체크(공개)

10. 외부 read-only 그래프 API (X-API-Key)

외부 기관/LLM용. 세 엔드포인트 모두 X-API-Key 인증(§1.2), 미설정 시 fail-closed.

엔드포인트	요청	응답(200)	설명
POST /api/v1/graph/read	cypher *, graph_path (기본 my_v40_demo), limit (기본 500, 최대 5000)	{graph_path, cypher, columns, row_count, rows}	read-only Cypher 실행 → 테이블
GET /api/v1/graph/dump	query graph_path *, limit (기본 500, 최대 5000), format (json triple)	json: {graph_path, node_count, edge_count, nodes, edges} / triple: {graph_path, triple_count, triples[[s,type,t]]}	그래프 전체 덤프(LLM 컨텍스트용)
GET /api/v1/graph/schema	query graph_path *	{graph_path, node_labels[], edge_types[], total_nodes, total_edges}	스키마 요약 (라벨/엣지 카운트)

read-only 강제: /read 는 사용자 Cypher를 받으므로

_WRITE_PATTERNS (CREATE|DELETE|MERGE|SET|DETACH|DROP|UPDATE|INSERT|ALTER|TRUNCATE|REMOVE|CALL) 정규식으로 차단(주석 제거 후 검사 → 우회 방지), 위반 시 403. /dump · /schema 는 서버 고정 쿼리만 실행. graph_path는 전부 ^[a-zA-Z_][a-zA-Z0-9_]*\$ 화이트리스트 검증.

11. 인증 예시 (curl)

```
export HOST="https://<HOST>"
export KEY="ccop_xxxxxxx"          # 기관별 Bearer 키
export RTOKEN="<운영팀 제공 조회토큰>" # X-API-Key (LLM_API_KEY와 일치)

# [Bearer] 자연어 → Cypher
curl -s -X POST $HOST/api/v1/text-to-cypher \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"question":"피의자2의 계좌를 보여줘"}'

# [Bearer] Cypher 실행 (읽기 전용, 티어 max_results 적용)
curl -s -X POST $HOST/api/v1/graph-query \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"cypher":"MATCH (v:vt_flnm) RETURN v LIMIT 10","graph_path":"tccop_graph_v6"}'

# [Bearer] 법률 근거 검색
curl -s -X POST $HOST/api/v1/legal/search \
  -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"question":"대포통장 양도 처벌","top_k":5,"mode":"hybrid"}'

# [X-API-Key] 외부 read-only Cypher
curl -s -X POST $HOST/api/v1/graph/read \
  -H "X-API-Key: $RTOKEN" -H "Content-Type: application/json" \
  -d '{"cypher":"MATCH (n) RETURN n LIMIT 10","graph_path":"tccop_graph_v6"}'

# [X-API-Key] 스키마 요약 / triple 덤프
curl -s -H "X-API-Key: $RTOKEN" "$HOST/api/v1/graph/schema?graph_path=tccop_graph_v6"
curl -s -H "X-API-Key: $RTOKEN" "$HOST/api/v1/graph/dump?graph_path=tccop_graph_v6&format=triple"
```

12. CORS · 보안 헤더 (`app/__init__.py`)

- CORS: `CORS_ORIGINS` env(콤마 구분, 기본 `http://localhost:5002`). * 운영 금지
- 보안 헤더: `X-Content-Type-Options: nosniff` , `X-Frame-Options: SAMEORIGIN` , `X-XSS-Protection` , `Referrer-Policy` , CSP(`frame-ancestors 'none'`), 프로덕션 시 `Strict-Transport-Security`
- 선택 Basic Auth: `BASIC_AUTH_USER/PASS` 설정 시 `/api/v1/health` 제외 전 경로 앞단 보호(무인증 엔드포인트도 이 층으로 커버 가능)
- gzip: 2xx·1KB↑·JSON/text 응답 압축

13. 개선 필요 (문서화 중 발견 — 후속 과제)

1. ✅ [해결] 무인증 파괴적 엔드포인트 — `graph/*` mutating 에 `@require_api_key` +admin 복구 (`fix/graph-admin-auth`)
2. ✅ [해결] enterprise `rate_limit=None` 비교 오류 — 무제한 분기 추가 (`fix/graph-admin-auth`)
3. ✅ [해결] UI-auth 정비 — `require_api_or_ui` (세션 OR Bearer) 도입, 하드코딩 `demo_key_123` 제거, network 기능 복구 (`fix/ui-auth-session`)
4. ⌚ Rate limit 단위 불일치 — 코드 주석("시간당")· `API_GUIDE.md` ("월") vs 실제 강제(분당) 정합
5. ⌚ 표준 에러 응답 헬퍼 도입(현재 `{error}` / `{error,message}` 혼재)
6. ⌚ 기존 `API_GUIDE.md` (5개)·외부 가이드와 본 레퍼런스의 단일화(중복 제거)

이 문서는 dev `58bf8b1` 코드 실측 기준. 엔드포인트 변경 시 함께 갱신할 것. 근거 `file:line` 은 조사 원본(`routes_api.py`, `routes_graph_read.py`, `api_auth.py`, `api_key.py`) 참조.